1) **Design and implement a Student Course Management System database using SQL.**

Create tables such as: Student (rollno, name, address, phno, email_id, marks) , Courses (c_id, cname, rollno) using CREATE, ALTER, RENAME, TRUNCATE and DROP tables where required. Also, apply constraints: PRIMARY KEY on student and course IDs ,FOREIGN KEY in Course table, NOT NULL for essential fields (e.g., student name) , UNIQUE for email and CHECK for valid marks (0–100) . Insert at least 7 to 10 records into each table. Demonstrate constraint enforcement by attempting invalid insertions. Use ON DELETE CASCADE

```
CREATE DATABASE student_course_db;

USE student_course_db;
CREATE TABLE Student (
   rollno INT PRIMARY KEY,
   name VARCHAR(50) NOT NULL,
   address VARCHAR(100),
   phno VARCHAR(15),
   email_id VARCHAR(50) UNIQUE,
   marks INT CHECK (marks >= 0 AND marks <= 100)
);
CREATE TABLE Courses (
   c_id INT PRIMARY KEY,
   cname VARCHAR(50),
   rollno INT,
   FOREIGN KEY (rollno) REFERENCES Student(rollno)
   ON DELETE CASCADE
);
ALTER TABLE Student ADD COLUMN age INT;
RENAME TABLE Courses TO Course_Details;
INSERT INTO Student VALUES
(1, 'Amit', 'Nagpur', '9876543210', 'amit@gmail.com', 85, 20),
```

(2, 'Riya', 'Mumbai', '9123456780', 'riya@gmail.com', 90, 21),

(3, 'Rahul', 'Pune', '9988776655', 'rahul@gmail.com', 75, 22),

(4, 'Sneha', 'Delhi', '8877665544', 'sneha@gmail.com', 88, 20),

(5, 'Karan', 'Chennai', '7766554433', 'karan@gmail.com', 92, 23),

(6, 'Neha', 'Bangalore', '6655443322', 'neha@gmail.com', 70, 21),

(7, 'Arjun', 'Hyderabad', '5544332211', 'arjun@gmail.com', 80, 22);

INSERT INTO Course_Details VALUES

(101, 'DBMS', 1),

(102, 'Java', 2),

(103, 'Python', 3),

(104, 'OS', 4),

(105, 'CN', 5),

(106, 'AI', 6),

(107, 'ML', 7);
INSERT INTO Student VALUES

(8, 'Test', 'Nagpur', '9999999999', 'test@gmail.com', 150, 20);
INSERT INTO Student VALUES

(9, 'Test2', 'Nagpur', '8888888888', 'amit@gmail.com', 80, 20);
INSERT INTO Student VALUES

(10, NULL, 'Nagpur', '7777777777', 'test2@gmail.com', 70, 20);
DELETE FROM Student WHERE rollno = 1;
SELECT * FROM Course_Details;
TRUNCATE TABLE Course_Details;
DROP TABLE Course_Details;

2) **Design and Implement following query using MongoDB**

Create a collection called „games, 2. Add 5 games to the database. Give each document the following properties: name, gametype, rating (out of 100), 3. Write a query that returns all the games, 4. Write a query that returns the 3 highest rated games, 5. Update your two favourite games to have two achievements called „Game Master" and Speed Demon", 6. Write a query that returns all the games that have both the „Game Maser" and 7. the „Speed Demon" achievements. 8. Write a query that returns only games that have achievements.

```
use gameDB
db.games.insertMany([
  { name: "BGMI", gametype: "Battle Royale", rating: 90 },
  { name: "Free Fire", gametype: "Battle Royale", rating: 85 },
  { name: "GTA V", gametype: "Adventure", rating: 95 },
  { name: "Minecraft", gametype: "Sandbox", rating: 92 },
  { name: "FIFA 23", gametype: "Sports", rating: 88 }
])
db.games.find()
db.games.find().sort({ rating: -1 }).limit(3)
db.games.updateOne(
  { name: "GTA V" },
  { $set: { achievements: ["Game Master", "Speed Demon"] } }
)


db.games.updateOne(
  { name: "Minecraft" },
  { $set: { achievements: ["Game Master", "Speed Demon"] } }
)
db.games.find({
  achievements: { $all: ["Game Master", "Speed Demon"] }
})
db.games.find({
  achievements: { $exists: true }
})
```

3)Create a table Employees (emp_id, emp_name, salary, dept_id) and Departments (dept_id, dept_name). Use Appropriate Keys
1. Insert at least **8–10 records** in the Employees table and **3–5 records** in Departments.
2. Write a query to display employees who earn **more than the average salary**.
3. Write a query to find the **employee(s) with the highest salary**.
4. Write a query to display employees whose salary is **greater than the average salary of their department** (correlated subquery).
5. Write a query to display employees working in departments where the **maximum salary is greater than 60,000**.
6. Write a query using **NOT IN** to find employees who are **not assigned to any department**.
7. Write a query using **EXISTS** to display employees who belong to departments located in a specific condition.

```
CREATE DATABASE company_db;
USE company_db;
CREATE TABLE Departments (
   dept_id INT PRIMARY KEY,
   dept_name VARCHAR(50)
);
CREATE TABLE Employees (
   emp_id INT PRIMARY KEY,
   emp_name VARCHAR(50),
   salary INT,
   dept_id INT,
   FOREIGN KEY (dept_id) REFERENCES Departments(dept_id)
);
INSERT INTO Departments VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance'),
(4, 'Marketing');
INSERT INTO Employees VALUES
(101, 'Amit', 50000, 1),
(102, 'Riya', 70000, 2),
(103, 'Rahul', 45000, 1),
(104, 'Sneha', 80000, 2),
(105, 'Karan', 60000, 3),
(106, 'Neha', 75000, 3),
(107, 'Arjun', 30000, NULL),
(108, 'Pooja', 90000, 2),
(109, 'Vikas', 65000, 4),
(110, 'Anjali', 40000, NULL);
SELECT * FROM Departments;
```

```
SELECT * FROM Employees;
SELECT * FROM Employees
WHERE salary > (SELECT AVG(salary) FROM Employees);
SELECT * FROM Employees
WHERE salary = (SELECT MAX(salary) FROM Employees);
SELECT * FROM Employees e
WHERE salary > (
   SELECT AVG(salary)
   FROM Employees
   WHERE dept_id = e.dept_id
);
SELECT * FROM Employees
WHERE dept_id IN (
   SELECT dept_id
   FROM Employees
   GROUP BY dept_id
   HAVING MAX(salary) > 60000
);
SELECT * FROM Employees
WHERE dept_id IS NULL;
SELECT * FROM Employees e
WHERE EXISTS (
   SELECT * FROM Departments d
   WHERE d.dept_id = e.dept_id
   AND d.dept_name = 'IT'
);
```

4)Design and implement a Student Course Management System database using SQL.

Create tables such as: Student (rollno, name, address, phno, email_id, marks) , Courses (c_id, cname, rollno) using CREATE, ALTER, RENAME, TRUNCATE and DROP tables where required. Also, apply constraints: PRIMARY KEY on student and course IDs ,FOREIGN KEY in Course table, NOT NULL for essential fields (e.g., student name) , UNIQUE for email and CHECK for valid marks (0–100) . Insert at least 7 to 10 records into each table. Demonstrate constraint enforcement by attempting invalid insertions. Use ON DELETE CASCADE

```
CREATE DATABASE student_course_db;

USE student_course_db;
CREATE TABLE Student (

   rollno INT PRIMARY KEY,
```

```
    name VARCHAR(50) NOT NULL,
    address VARCHAR(100),
    phno VARCHAR(15),
    email_id VARCHAR(50) UNIQUE,
    marks INT CHECK (marks >= 0 AND marks <= 100)
);
CREATE TABLE Courses (
    c_id INT PRIMARY KEY,
    cname VARCHAR(50),
    rollno INT,
    FOREIGN KEY (rollno) REFERENCES Student(rollno)
    ON DELETE CASCADE
);
ALTER TABLE Student ADD COLUMN age INT;
RENAME TABLE Courses TO Course_Details;
INSERT INTO Student VALUES
(1, 'Amit', 'Nagpur', '9876543210', 'amit@gmail.com', 85, 20),
(2, 'Riya', 'Mumbai', '9123456780', 'riya@gmail.com', 90, 21),
(3, 'Rahul', 'Pune', '9988776655', 'rahul@gmail.com', 75, 22),
(4, 'Sneha', 'Delhi', '8877665544', 'sneha@gmail.com', 88, 20),
(5, 'Karan', 'Chennai', '7766554433', 'karan@gmail.com', 92, 23),
(6, 'Neha', 'Bangalore', '6655443322', 'neha@gmail.com', 70, 21),
(7, 'Arjun', 'Hyderabad', '5544332211', 'arjun@gmail.com', 80, 22);

INSERT INTO Course_Details VALUES
(101, 'DBMS', 1),
(102, 'Java', 2),
(103, 'Python', 3),
(104, 'OS', 4),
(105, 'CN', 5),
```

```
(106, 'AI', 6),

(107, 'ML', 7);
INSERT INTO Student VALUES

(8, 'Test', 'Nagpur', '9999999999', 'test@gmail.com', 150, 20);
INSERT INTO Student VALUES

(9, 'Test2', 'Nagpur', '8888888888', 'amit@gmail.com', 80, 20);
INSERT INTO Student VALUES
(10, NULL, 'Nagpur', '7777777777', 'test2@gmail.com', 70, 20);
DELETE FROM Student WHERE rollno = 1;
SELECT * FROM Course_Details;
TRUNCATE TABLE Course_Details;
DROP TABLE Course_Details;
```

5)Develop a database for an Online Shopping System using SQL.
Create tables such as: Customer (c_id, name, address, phno, email) ,Product (p_id, pname, c_id, price, qty).
Using CREATE tables , ALTER to modify structure, RENAME any one table , TRUNCATE and DROP operations for testing . Also apply constraints: PRIMARY KEY for each table , FOREIGN KEY in product referencing Customers , NOT NULL for important fields (name, price) , UNIQUE for customer email or product code , CHECK for price > 0 and quantity ≥ 1
Insert sample data (minimum 7 to 10 records per table), Test integrity constraints by inserting incorrect data.ON DELETE CASCADE.

```
CREATE DATABASE shopping_db;

USE shopping_db;
CREATE TABLE Customer (

  c_id INT PRIMARY KEY,

  name VARCHAR(50) NOT NULL,

  address VARCHAR(100),

  phno VARCHAR(15),

  email VARCHAR(50) UNIQUE

);
CREATE TABLE Product (

  p_id INT PRIMARY KEY,
```

```
    pname VARCHAR(50),
    c_id INT,
    price INT CHECK (price > 0),
    qty INT CHECK (qty >= 1),
    FOREIGN KEY (c_id) REFERENCES Customer(c_id)
    ON DELETE CASCADE
);
ALTER TABLE Product ADD COLUMN category VARCHAR(30);
RENAME TABLE Product TO Products;
INSERT INTO Customer VALUES
(1, 'Amit', 'Nagpur', '9876543210', 'amit@gmail.com'),
(2, 'Riya', 'Mumbai', '9123456780', 'riya@gmail.com'),
(3, 'Rahul', 'Pune', '9988776655', 'rahul@gmail.com'),
(4, 'Sneha', 'Delhi', '8877665544', 'sneha@gmail.com'),
(5, 'Karan', 'Chennai', '7766554433', 'karan@gmail.com'),
(6, 'Neha', 'Bangalore', '6655443322', 'neha@gmail.com'),
(7, 'Arjun', 'Hyderabad', '5544332211', 'arjun@gmail.com');

INSERT INTO Products VALUES
(101, 'Laptop', 1, 50000, 2, 'Electronics'),
(102, 'Mobile', 2, 20000, 5, 'Electronics'),
(103, 'Shoes', 3, 3000, 3, 'Fashion'),
(104, 'Watch', 4, 2500, 4, 'Accessories'),
(105, 'Bag', 5, 1500, 6, 'Fashion'),
(106, 'Headphones', 6, 2000, 2, 'Electronics'),
(107, 'Keyboard', 7, 1200, 3, 'Electronics');

SELECT * FROM Customer;

SELECT * FROM Products;
INSERT INTO Products VALUES
```

(108, 'InvalidItem', 1, -500, 2, 'Test');
INSERT INTO Products VALUES

(109, 'InvalidQty', 2, 1000, 0, 'Test');
INSERT INTO Customer VALUES
(8, 'Test', 'Nagpur', '9999999999', 'amit@gmail.com');
DELETE FROM Customer WHERE c_id = 1;
SELECT * FROM Products;
TRUNCATE TABLE Products;
DROP TABLE Products;

6) A MongoDB collection named orders contains documents with the following structure:
Orders(Id, name, product, category, amt, date)
1) Find the total sales amount for each category.
2) Find the average order amount per category.
3) Sort categories by total sales in descending order.

```
db.orders.insertMany([
 { _id: 1, name: "Amit", product: "Laptop", category: "Electronics", amt: 50000, date: "2024-01-01" },
 { _id: 2, name: "Riya", product: "Mobile", category: "Electronics", amt: 20000, date: "2024-01-02" },
 { _id: 3, name: "Rahul", product: "Shoes", category: "Fashion", amt: 3000, date: "2024-01-03" },
 { _id: 4, name: "Sneha", product: "Dress", category: "Fashion", amt: 4000, date: "2024-01-04" },
 { _id: 5, name: "Karan", product: "TV", category: "Electronics", amt: 30000, date: "2024-01-05" }
])
db.orders.aggregate([
 {
   $group: {
    _id: "$category",
    total_sales: { $sum: "$amt" }
   }
 }
])
db.orders.aggregate([
```

```
 {
  $group: {
   _id: "$category",
   avg_sales: { $avg: "$amt" }
  }
 }
])
db.orders.aggregate([
 {
  $group: {
   _id: "$category",
   total_sales: { $sum: "$amt" }
  }
 },
 {
  $sort: { total_sales: -1 }
 }
])
```

7) A MongoDB collection named orders contains documents with the following structure:
Orders(Id, name, product, category, amt, date)
1) Find the total sales amount for each category.
2) Find the average order amount per category.
3) Sort categories by total sales in descending order.

```
db.orders.insertMany([
 { _id: 1, name: "Amit", product: "Laptop", category: "Electronics", amt: 50000, date: "2024-01-01" },
 { _id: 2, name: "Riya", product: "Mobile", category: "Electronics", amt: 20000, date: "2024-01-02" },
 { _id: 3, name: "Rahul", product: "Shoes", category: "Fashion", amt: 3000, date: "2024-01-03" },
```

```
 { _id: 4, name: "Sneha", product: "Dress", category: "Fashion", amt: 4000, date: "2024-01-04" },
 { _id: 5, name: "Karan", product: "TV", category: "Electronics", amt: 30000, date: "2024-01-05" }
])
db.orders.aggregate([
 {
  $group: {
   _id: "$category",
   total_sales: { $sum: "$amt" }
  }
 }
])
db.orders.aggregate([
 {
  $group: {
   _id: "$category",
   avg_sales: { $avg: "$amt" }
  }
 }
])
db.orders.aggregate([
 {
  $group: {
   _id: "$category",
   total_sales: { $sum: "$amt" }
  }
 },
 {
  $sort: { total_sales: -1 }
 }
])
```

. **8) Improve Query Performance using Indexing, Consider a collection users: (Id, name, mailed, age, city)** 1. Create an index on the email field to improve search performance.
2. Create a compound index on city and age.
3. Query users from Mumbai aged above 20 efficiently

```
use userDB
db.users.insertMany([
  { name: "Amit", email: "amit@gmail.com", age: 25, city: "Mumbai" },
  { name: "Riya", email: "riya@gmail.com", age: 30, city: "Pune" },
  { name: "Rahul", email: "rahul@gmail.com", age: 22, city: "Mumbai" },
  { name: "Sneha", email: "sneha@gmail.com", age: 19, city: "Mumbai" },
  { name: "Karan", email: "karan@gmail.com", age: 35, city: "Delhi" }
])
db.users.createIndex({ email: 1 })
db.users.createIndex({ city: 1, age: 1 })
db.users.find({
  city: "Mumbai",
  age: { $gt: 20 }
})
```

9)SQL Queries for Data Manipulation, Access Control, and Transactions Design and run SQL queries to demonstrate the following:
Data Manipulation (DML): Use SQL statements to INSERT, UPDATE, and DELETE records. Apply arithmetic, logical, set operators, pattern matching, and string functions.

```
CREATE DATABASE demo_db;
USE demo_db;
CREATE TABLE Student (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    marks INT,
    city VARCHAR(50)
);
INSERT INTO Student VALUES
```

```
(1, 'Amit', 80, 'Pune'),

(2, 'Riya', 90, 'Mumbai'),

(3, 'Rahul', 70, 'Nagpur'),

(4, 'Sneha', 85, 'Pune');
UPDATE Student

SET marks = marks + 5

WHERE name = 'Amit';
DELETE FROM Student

WHERE id = 3;
SELECT * FROM Student

WHERE marks > 80 AND city = 'Pune';
SELECT name FROM Student WHERE city = 'Pune'

UNION

SELECT name FROM Student WHERE marks > 85;
SELECT * FROM Student

WHERE name LIKE 'A%';
SELECT UPPER(name), LENGTH(name)
FROM Student;
CREATE USER 'user1'@'localhost' IDENTIFIED BY '1234';
GRANT SELECT, INSERT ON Student TO 'user1'@'localhost';
REVOKE INSERT ON Student FROM 'user1'@'localhost';
START TRANSACTION;
UPDATE Student SET marks = 100 WHERE id = 1;
COMMIT;
ROLLBACK;
```

10)Develop a Library Management System database to manage books and members. Create tables:
a. Members (member_id, name, email) , Books (book_id, title, author, price)
b. Borrow (borrow_id, member_id, book_id, date) ,

Perform: INSERT records into all tables , UPDATE book price or member details , DELETE records for returned/lost books. Use:Arithmetic operations (price calculation with discount) , Logical conditions (price > 500 AND author='XYZ') , Pattern matching (title LIKE '%SQL%') , String functions (LOWER(), CONCAT()) , Set operators (INTERSECT or UNION for reports).

```
CREATE DATABASE library_db;
```

```
USE library_db;
CREATE TABLE Members (
   member_id INT PRIMARY KEY,
   name VARCHAR(50),
   email VARCHAR(50)
);
CREATE TABLE Books (
   book_id INT PRIMARY KEY,
   title VARCHAR(100),
   author VARCHAR(50),
   price INT
);
CREATE TABLE Borrow (
   borrow_id INT PRIMARY KEY,
   member_id INT,
   book_id INT,
   date DATE,
   FOREIGN KEY (member_id) REFERENCES Members(member_id),
   FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
INSERT INTO Members VALUES
(1, 'Amit', 'amit@gmail.com'),
(2, 'Riya', 'riya@gmail.com'),
(3, 'Rahul', 'rahul@gmail.com');
INSERT INTO Books VALUES
(101, 'SQL Basics', 'XYZ', 600),
(102, 'DBMS Guide', 'ABC', 400),
(103, 'Advanced SQL', 'XYZ', 800),
(104, 'Java Book', 'DEF', 500);
INSERT INTO Borrow VALUES
(1, 1, 101, '2024-01-01'),
(2, 2, 102, '2024-02-01'),
```

```
(3, 3, 103, '2024-03-01');
UPDATE Books

SET price = price + 50

WHERE book_id = 102;
DELETE FROM Books

WHERE book_id = 104;
SELECT title, price, price * 0.9 AS discounted_price

FROM Books;
SELECT * FROM Books

WHERE price > 500 AND author = 'XYZ';
SELECT * FROM Books

WHERE title LIKE '%SQL%';
SELECT LOWER(title), CONCAT(title, ' - ', author)

FROM Books;
SELECT title FROM Books WHERE price > 500

UNION
SELECT title FROM Books WHERE author = 'XYZ';
```

11)Develop a database to analyze sales performance of a company.

Create a table: Sales (sale_id, product_name, region, quantity, price)
Insert at least 10–15 records covering multiple products and regions.
Write SQL queries to:
1. Calculate total sales amount (SUM(quantity * price)) for each product.
2. Find average sales per region using AVG().
3. Identify highest and lowest sales using MAX() and MIN().
4. Use GROUP BY to group data by product_name and region.
5. Use HAVING to:
- Display products with total sales > 5000
- Show regions where average sales < 1000

```
CREATE DATABASE sales_db;

USE sales_db;
CREATE TABLE Sales (

  sale_id INT PRIMARY KEY,

  product_name VARCHAR(50),

  region VARCHAR(50),

  quantity INT,

  price INT
```

```
);
INSERT INTO Sales VALUES
(1, 'Laptop', 'Mumbai', 5, 50000),
(2, 'Mobile', 'Pune', 10, 20000),
(3, 'Laptop', 'Nagpur', 3, 50000),
(4, 'Tablet', 'Mumbai', 7, 15000),
(5, 'Mobile', 'Nagpur', 8, 20000),
(6, 'Laptop', 'Pune', 4, 50000),
(7, 'Tablet', 'Pune', 6, 15000),
(8, 'Mobile', 'Mumbai', 9, 20000),
(9, 'Laptop', 'Mumbai', 2, 50000),
(10, 'Tablet', 'Nagpur', 5, 15000),
(11, 'Mobile', 'Pune', 7, 20000),
(12, 'Laptop', 'Nagpur', 6, 50000);
SELECT product_name,
SUM(quantity * price) AS total_sales
FROM Sales
GROUP BY product_name;
SELECT region,
AVG(quantity * price) AS avg_sales
FROM Sales
GROUP BY region;
SELECT
MAX(quantity * price) AS highest_sales,
MIN(quantity * price) AS lowest_sales
FROM Sales;
SELECT product_name, region,
SUM(quantity * price) AS total_sales
FROM Sales
GROUP BY product_name, region;
SELECT product_name,
SUM(quantity * price) AS total_sales
```

```
FROM Sales

GROUP BY product_name

HAVING total_sales > 5000;
SELECT region,

AVG(quantity * price) AS avg_sales

FROM Sales

GROUP BY region

HAVING avg_sales < 1000;
```

12)CRUD Operations using MongoDB

Design and implement basic Create, Read, Update, and Delete (CRUD) operations using MongoDB. Use the save method and logical operators wherever necessary.

```
use practicaldb
db.students.insertOne({ _id: 1, name: "Amit", age: 20, city: "Pune" })
db.students.insertMany([

 { _id: 2, name: "Riya", age: 22, city: "Mumbai" },

 { _id: 3, name: "Rahul", age: 19, city: "Nagpur" }

])
db.students.find()
db.students.find({

 $and: [

  { age: { $gt: 20 } },

  { city: "Mumbai" }

 ]

})
db.students.updateOne(

 { _id: 2 },

 { $set: { age: 23 } }
)
db.students.find()
db.students.deleteOne({ _id: 3 })
db.students.find()
```

13)CRUD Operations using MongoDB
Design and implement basic Create, Read, Update, and Delete (CRUD) operations using MongoDB. Use the save method and logical operators wherever necessary.

```
use practicaldb
db.students.insertOne({ _id: 1, name: "Amit", age: 20, city: "Pune" })
db.students.insertMany([
 { _id: 2, name: "Riya", age: 22, city: "Mumbai" },
 { _id: 3, name: "Rahul", age: 19, city: "Nagpur" }
])
db.students.find()
db.students.find({
 $and: [
  { age: { $gt: 20 } },
  { city: "Mumbai" }
 ]
})
db.students.updateOne(
 { _id: 2 },
 { $set: { age: 23 } }
)
db.students.find()
db.students.deleteOne({ _id: 3 })
db.students.find()
```

14)Solve Below Queries: Use Set operations, Tuple Variable and Order by clause
1. Find all customers who have a loan, an account, or both:
2. Find all customers who have both a loan and an account.
3. Find all customers who have an account but no loan.
4. Find the customer names and their loan numbers and amount for all customers having a loan at some branch using tuple variable
5. Find the names of all branches that have greater assets than some branch located in Brooklyn using tuple variable
6. List in alphabetic order the names of all customers having a loan in Perryridge branch

Use Depositor and borrower relation with appropriate data" Create table and insert 5 rows.

```
CREATE DATABASE bank_set_db;
```

```
USE bank_set_db;
CREATE TABLE Depositor (
   cname VARCHAR(50),
   account_no INT
);
CREATE TABLE Borrower (
   cname VARCHAR(50),
   loan_no INT,
   amount INT,
   branch VARCHAR(50)
);
CREATE TABLE Branch (
   branch_name VARCHAR(50),
   assets INT,
   city VARCHAR(50)
);
INSERT INTO Depositor VALUES
('Amit', 101),
('Riya', 102),
('Rahul', 103),
('Sneha', 104),
('Karan', 105);
INSERT INTO Borrower VALUES
('Amit', 201, 5000, 'Perryridge'),
('Riya', 202, 3000, 'Perryridge'),
('Neha', 203, 7000, 'Brooklyn'),
('Rahul', 204, 2000, 'Perryridge'),
('Sunil', 205, 9000, 'Mumbai');
INSERT INTO Branch VALUES
('Perryridge', 50000, 'NY'),
('Brooklyn', 40000, 'NY'),
('Mumbai', 60000, 'India'),
```

```
('Delhi', 55000, 'India'),

('Chicago', 45000, 'USA');
SELECT cname FROM Depositor

UNION

SELECT cname FROM Borrower;
SELECT cname

FROM Depositor

WHERE cname IN (

  SELECT cname FROM Borrower

);
SELECT cname FROM Depositor

WHERE cname NOT IN (SELECT cname FROM Borrower);
SELECT b.cname, b.loan_no, b.amount

FROM Borrower b;
SELECT b1.branch_name

FROM Branch b1

WHERE b1.assets > (

  SELECT b2.assets

  FROM Branch b2

  WHERE b2.branch_name = 'Brooklyn'

);
SELECT cname

FROM Borrower

WHERE branch = 'Perryridge'
ORDER BY cname ASC;
(((
```

15)Database Triggers Implement and test triggers to maintain data integrity in database Use appropriate data and implement Before and after

```
CREATE DATABASE trigger_db;

USE trigger_db;
CREATE TABLE Student (
```

```
  id INT PRIMARY KEY,
  name VARCHAR(50),
  marks INT
);
CREATE TABLE Student_Log (
  id INT,
  message VARCHAR(50)
);

DELIMITER //

CREATE TRIGGER before_marks_check
BEFORE INSERT ON Student
FOR EACH ROW
BEGIN
  IF NEW.marks > 100 THEN
    SET NEW.marks = 100;
  END IF;
END //

DELIMITER ;

DELIMITER //

CREATE TRIGGER after_insert_log
AFTER INSERT ON Student
FOR EACH ROW
BEGIN
  INSERT INTO Student_Log VALUES (NEW.id, 'Record Inserted');
END //
```

```
DELIMITER ;
INSERT INTO Student VALUES (1, 'Amit', 150);
INSERT INTO Student VALUES (2, 'Riya', 90);
SELECT * FROM Student;
SELECT * FROM Student_Log;
```

16)Create a db called company consist of the following tables.

1.Emp (eno,ename, job,hiredate,salary,commission,deptno,)
2.dept(deptno,deptname,location)
eno is primary key in emp
deptno is primary key in dept
Solve Queries by SQL
1. List the maximum salary paid to salesman
2. List name of emp whose name start with „I"
3. List the emp details in the descending order of their basic salary
4. List the avg salary, minimum salary of the emp hire datewise for dept no „10".
5. List emp name and its department
6. List total salary paid to each department
7. List details of employee working in „Dev" department.

```
CREATE DATABASE company;

USE company;
CREATE TABLE dept (

  deptno INT PRIMARY KEY,

  deptname VARCHAR(50),

  location VARCHAR(50)

);
CREATE TABLE emp (

  eno INT PRIMARY KEY,

  ename VARCHAR(50),

  job VARCHAR(50),

  hiredate DATE,

  salary INT,

  commission INT,

  deptno INT,
```

```
  FOREIGN KEY (deptno) REFERENCES dept(deptno)
);
INSERT INTO dept VALUES
(10, 'Dev', 'Mumbai'),
(20, 'HR', 'Pune'),
(30, 'Sales', 'Delhi');
INSERT INTO emp VALUES
(1, 'Amit', 'Salesman', '2022-01-10', 30000, 2000, 30),
(2, 'Isha', 'Developer', '2021-03-15', 50000, 0, 10),
(3, 'Rahul', 'Salesman', '2020-06-20', 40000, 3000, 30),
(4, 'Irfan', 'Manager', '2019-02-11', 60000, 0, 20),
(5, 'Neha', 'Developer', '2023-07-01', 45000, 0, 10);
SELECT MAX(salary)
FROM emp
WHERE job = 'Salesman';
SELECT * FROM emp
WHERE ename LIKE 'I%';
SELECT * FROM emp
ORDER BY salary DESC;
SELECT hiredate,
AVG(salary) AS avg_salary,
MIN(salary) AS min_salary
FROM emp
WHERE deptno = 10
GROUP BY hiredate;
SELECT e.ename, d.deptname
FROM emp e, dept d
WHERE e.deptno = d.deptno;
SELECT d.deptname, SUM(e.salary) AS total_salary
FROM emp e, dept d
WHERE e.deptno = d.deptno
GROUP BY e.deptno;
SELECT e.ename
```

```
FROM emp e, dept d

WHERE e.deptno = d.deptno

AND d.deptname = 'Dev';
```